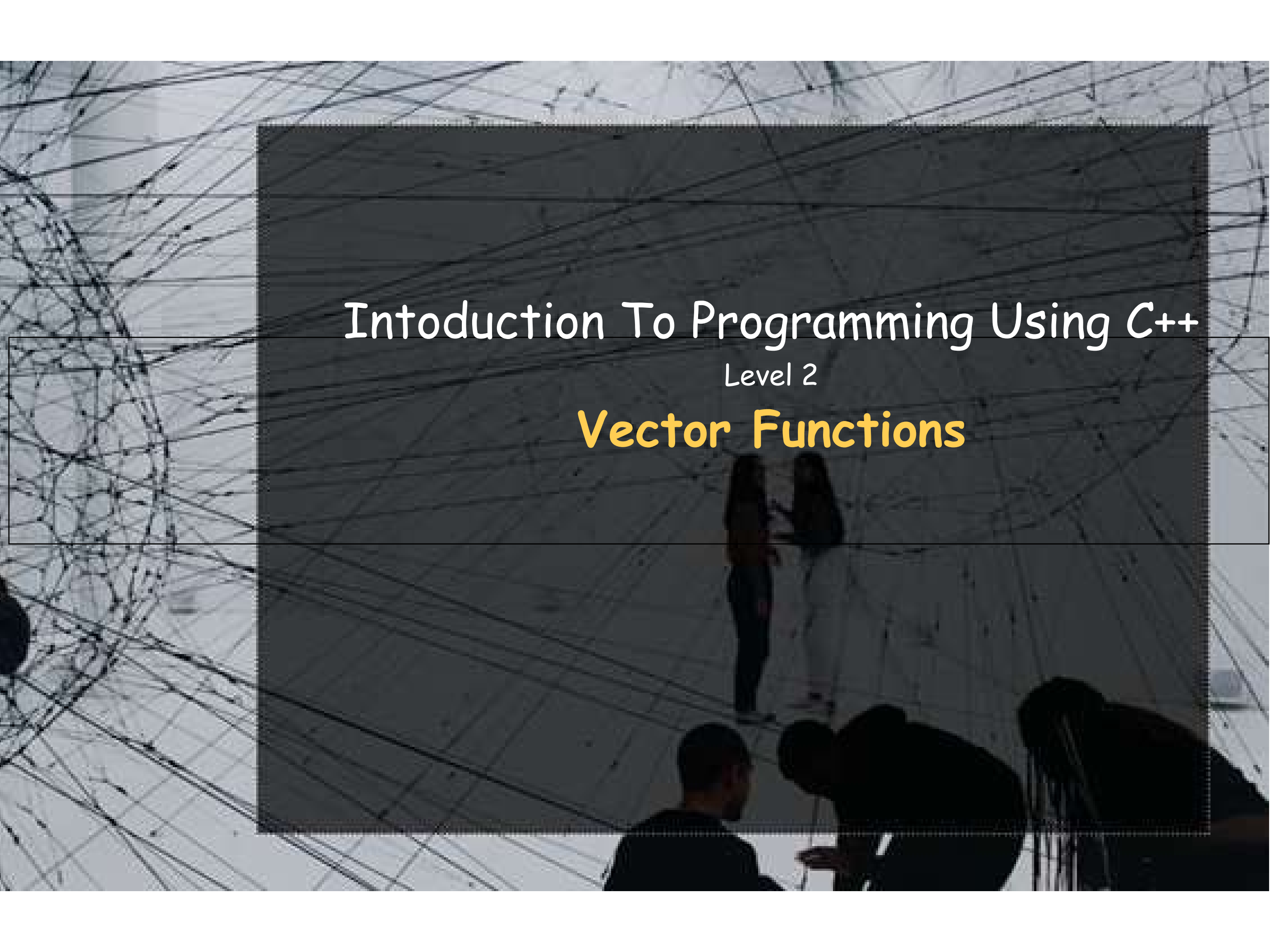




Introduction To Programming Using C++ Level 2

Vector Functions



Introduction To Programming Using C++

Level 2

Vector Functions

Definition

- ◆ In C++, a vector is a dynamic array that provides several ready-made member functions for managing its elements.
- ◆ These functions make it easier to add, remove, access, and check elements, as well as inspect the vector's size and capacity.
- ◆ The main functions covered in this lesson are `clear()`, `front()`, `back()`, `size()`, `capacity()`, and `empty()`.

Explanation

- ◆ `vector<int> vNumbers;` creates an empty vector of integers.
- ◆ Elements can be added using `push_back()`.
- ◆ Once elements are stored, vector member functions can be used to access information about the vector.
- ◆ `clear()` removes all elements from the vector.
- ◆ `front()` returns the first element.
- ◆ `back()` returns the last element.
- ◆ `size()` returns the current number of elements.
- ◆ `capacity()` returns the amount of storage currently allocated for the vector.
- ◆ `empty()` checks whether the vector contains any elements.

Procedures

1. Create a vector.
2. Add elements using `push_back()`.
3. Use `front()` to access the first element.
4. Use `back()` to access the last element.
5. Use `size()` to get the number of current elements.
6. Use `capacity()` to check the allocated capacity.
7. Use `empty()` to check whether the vector is empty.
8. Use `clear()` when you want to remove all elements.

💡 Example

- ◆ The vector contains five elements:

```
vector<int> vNumbers;  
vNumbers.push_back(10);  
vNumbers.push_back(20);  
vNumbers.push_back(30);  
vNumbers.push_back(40);  
vNumbers.push_back(50);  
cout << "First Element: " << vNumbers.front() << endl;  
cout << "Last Element: " << vNumbers.back() << endl;  
cout << "Size of Vector: " << vNumbers.size() << endl;  
cout << "Capacity of Vector: " << vNumbers.capacity() << endl;  
cout << "If vector is empty print true/1 : " << vNumbers.empty() << endl;
```

➡ Result

```
First Element: 10  
Last Element: 50  
Size of Vector: 5  
Capacity of Vector: 8  
If vector is empty print true/1 : 0
```

✈ Note

- ◆ `size()` represents the actual number of elements currently stored in the vector.
- ◆ `capacity()` represents the amount of storage currently allocated for the vector.
- ◆ Therefore, `capacity()` can be greater than `size()`.
- ◆ For example, the vector can have `size() = 5` while `capacity() = 8`.
- ◆ After using `clear()`, the vector contains no elements.
- ◆ `clear()` removes the elements, but it does not necessarily reduce the vector's capacity.
- ◆ Before accessing the first or last element, make sure the vector is not empty.
- ◆ Before removing an element with `pop_back()`, make sure the vector contains at least one element.
- ◆ You can check this using:

```
if (vNumbers.size() > 0)
    vNumbers.pop_back();
```
- ◆ Or:

```
if (!vNumbers.empty())
    vNumbers.pop_back();
```

⊘ Warning

- ◆ Do not use `front()` or `back()` on an empty vector because the operation is invalid.
- ◆ Do not use `pop_back()` on an empty vector.
- ◆ `clear()` removes all elements, so the existing elements can no longer be accessed through the vector.
- ◆ The exact value returned by `capacity()` can depend on the implementation and the vector's growth strategy.
- ◆ Do not assume that `capacity()` will always exactly double.
- ◆ When modifying a vector during iteration, be careful because removing or clearing elements can invalidate iterators and change the vector's contents.

Comparison

size()

- ✓ Returns the actual number of elements currently stored.
- ✓ Example: five elements → size() returns 5.
- ✓ Changes when elements are added or removed.

capacity()

- ✓ Returns the amount of storage currently allocated.
- ✓ Can be larger than the current number of elements.
- ✓ Example: size() = 5, capacity() = 8.

front()

- ✓ Returns the first element.
- ✓ Example: 10 20 30 → front() returns 10.
- ✓ The vector must not be empty.

back()

- ✓ Returns the last element.
- ✓ Example: 10 20 30 → back() returns 30.
- ✓ The vector must not be empty.

Comparison

clear()

- ✓ Removes all elements.
- ✓ The vector becomes empty.
- ✓ Does not necessarily reduce its capacity.

empty()

- ✓ Checks whether the vector contains zero elements.
- ✓ Returns true (1) when empty.
- ✓ Returns false (0) when it contains elements.

✦ Summary

- ✓ `clear()` removes all elements from the vector.
- ✓ `front()` returns the first element.
- ✓ `back()` returns the last element.
- ✓ `size()` returns the current number of elements.
- ✓ `capacity()` returns the currently allocated storage capacity.
- ✓ `empty()` checks whether the vector is empty.
- ✓ `size()` and `capacity()` are different: `size` is the number of stored elements, while `capacity` is the allocated storage.
- ✓ Always check that the vector is not empty before using `front()`, `back()`, or `pop_back()`.

Thank you 😊